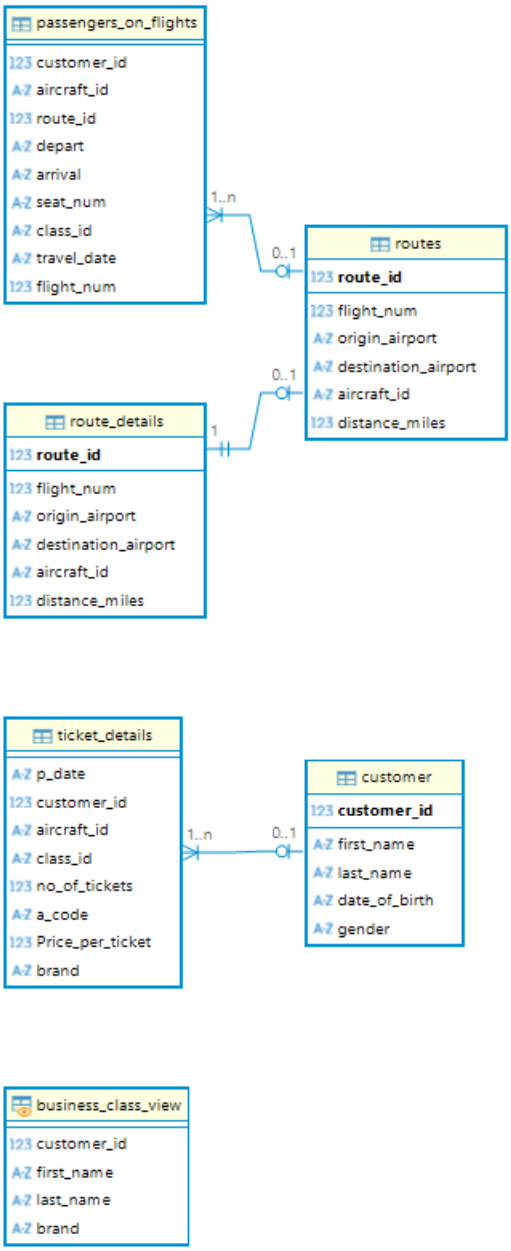
Course-End Project: Air Cargo Analysis

Project Objective: Analyze Customer Demographics, routes and ticket sales to improve operability and customer-centricity using SQL

ER Diagram



1. **Create the Route Details Table**

```
CREATE TABLE `route_details` (
  `route_id` int DEFAULT NULL,
  `flight_num` int DEFAULT NULL,
  `origin_airport` varchar(50) DEFAULT NULL,
  `destination_airport` varchar(50) DEFAULT NULL,
  `aircraft_id` varchar(50) DEFAULT NULL,
  `distance_miles` int DEFAULT NULL,
  UNIQUE KEY `route_id` (`route_id`),
  CONSTRAINT `route_details_chk_1` CHECK ((`flight_num` is not
null)),
  CONSTRAINT `route_details_chk_2` CHECK ((`distance_miles` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;
```

2. **Passengers on Specific Routes**

```
SELECT DISTINCT customer_id
FROM passengers_on_flights
WHERE route_id BETWEEN 1 AND 25;
```

passengers_on_flights 1

Enter a SQL expression to filt

	123 customer_id
1	2
2	1
3	5
4	4
5	7
6	11
7	17
8	9
9	10
10	15
11	13
12	22
13	24
14	25
15	50
16	29
17	44
18	46
19	49
20	31
21	18

3. Number of passengers and total revenue in business class

```
SELECT COUNT(customer_id) AS number_of_passengers,
       SUM(price_per_ticket * no_of_tickets) AS total_revenue
FROM ticket_details
WHERE class_id = 'Business';
```

ticket_details 1

Enter a SQL expression to filter results (use Ctrl+Space)

	123 number_of_passengers	123 total_revenue
1	0	[NULL]

4. Full name of the customer

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM customer;
```

customer 1

Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z full_name
1	Julie Sam
2	Steve Ryan
3	Morris Lois
4	Cathenna Emily
5	Aaron Kim
6	Alexander Scot
7	Anderson Stewart
8	Floyd Ted
9	Leo Travis
10	Melvin Tracy
11	Roger Walson
12	Shirley Wally
13	Solomon Walter
14	Carol Vernon
15	Linda William
16	Chirstine Willis
17	Catherine Shad
18	Gloria Richie
19	Joyce Paul
20	Sara Oliver
21	Chirsty Josh
22	Pheny Eri
23	Erwin Tosh
24	Calvin Willis
25	Moss Morris
26	Bryan Collin
27	Cheriy Vernon

5. **Customers who have registered and booked a ticket**

```
FROM customer c
INNER JOIN ticket_details t ON c.customer_id = t.customer_id;
```

customer 1

Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	first_name	last_name
4	4	Cathenna	Emily
5	5	Aaron	Kim
6	7	Anderson	Stewart
7	8	Floyd	Ted
8	9	Leo	Travis
9	10	Melvin	Tracy
10	11	Roger	Walson
11	19	Joyce	Paul
12	13	Solomon	Walter

6. **Customers based on brand (Emirates)**

```
FROM customer c
JOIN ticket_details t ON c.customer_id = t.customer_id
WHERE t.brand = 'Emirates';
```

customer 1

Enter a SQL expression to filter results (use Ctrl+Space)

	first_name	last_name
1	Steve	Ryan
2	Cathenna	Emily
3	Cathenna	Emily
4	Aaron	Kim
5	Anderson	Stewart
6	Leo	Travis
7	Roger	Walson
8	Roger	Walson
9	Carol	Vernon
10	Gloria	Richie
11	Gloria	Richie
12	Joyce	Paul
13	Moss	Morris
14	Moss	Morris
15	Cherly	Vernon
16	James	Robert
17	Bily	Brian
18	Russell	Peter

7. **Customers in Economy Plus (Group By & Having)**

```
SELECT customer_id
FROM passengers_on_flights
GROUP BY customer_id, class_id
HAVING class_id = 'Economy Plus';
```

passengers_on_flights 1

SELECT customer_id FROM passengers_on_flights GROUP BY | Enter a SQL expression to filter

	123 customer_id
1	1
2	8
3	11
4	17
5	19
6	22
7	32
8	47
9	50

8. **Check if total revenue crossed 10000**

```
SELECT SUM(price_per_ticket * no_of_tickets) AS Total_Revenue,
       IF(SUM(price_per_ticket * no_of_tickets) > 10000, 'Crossed 10000', 'Not Crossed 10000') AS Revenue_Status
FROM ticket_details;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

	123 Total_Revenue	A-Z Revenue_Status
1	15,369	Crossed 10000

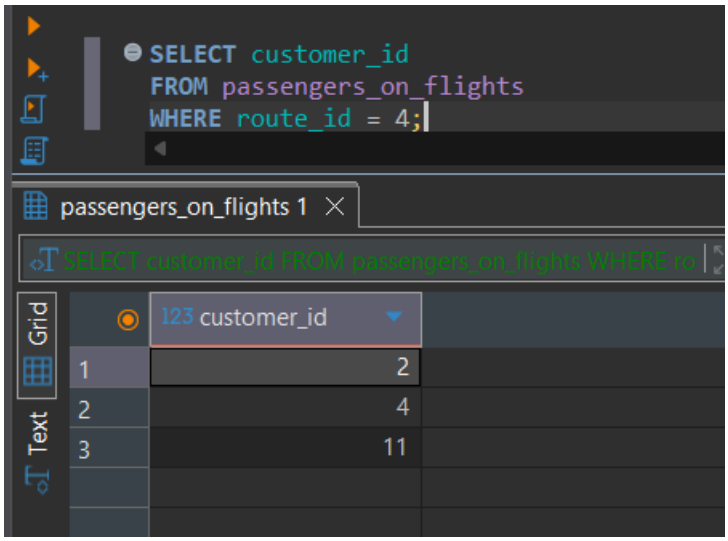
9. **Create and grant access to a new DBA user**

```
CREATE USER 'new_dba_user'@'localhost' IDENTIFIED BY 'SecurePass123!';
GRANT ALL PRIVILEGES ON aircargo_db.* TO 'new_dba_user'@'localhost';
FLUSH PRIVILEGES;
```

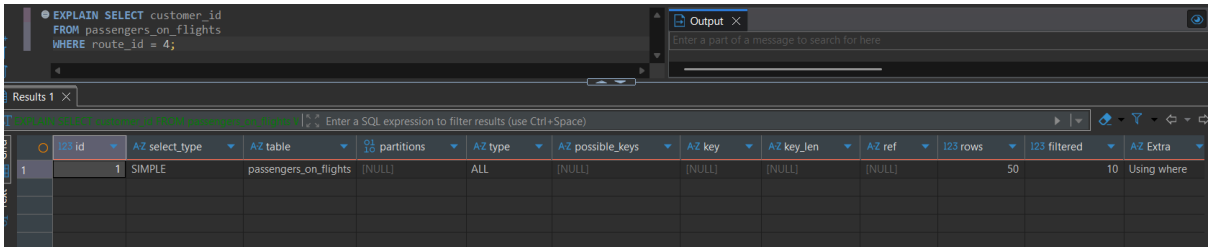
Statistics 1

Name	Value
Updated Rows	0
Execute time	0.05s
Start time	Tue Jun 09 06:27:19 SAST 2026
Finish time	Tue Jun 09 06:27:19 SAST 2026
Query	FLUSH PRIVILEGES

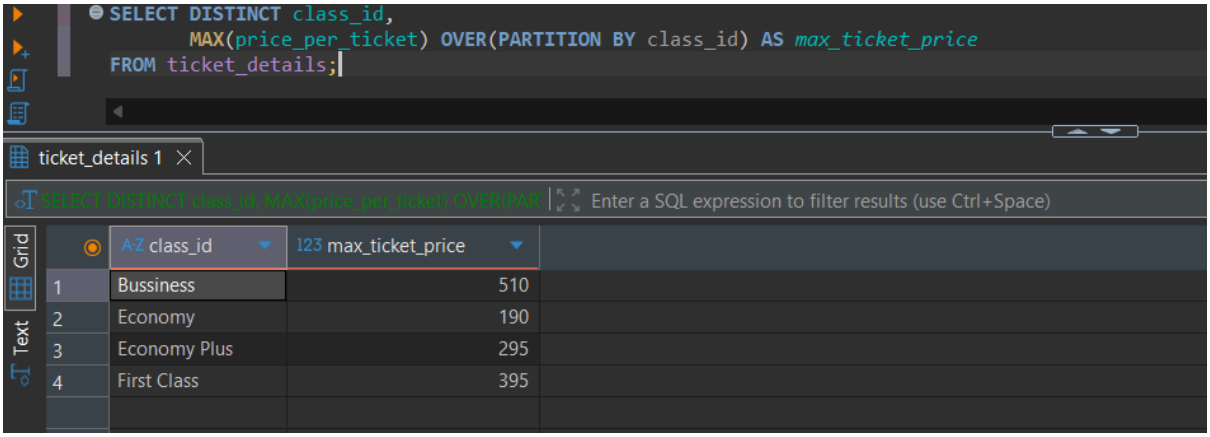
10. Improve speed and extract passengers for Route ID 4



11. View the execution plan for Route ID 4



12. **Find maximum ticket price for each class (Window Function)**



13. **Calculate total price across different aircraft IDs (Rollup)**

```
SELECT customer_id, aircraft_id, SUM(price_per_ticket * no_of_tickets) AS total_price
FROM ticket_details
GROUP BY customer_id, aircraft_id WITH ROLLUP;
```

Results 1

Enter a SQL expression to filter results (Ctrl+Space)

	customer_id	aircraft_id	total_price
1	1	CRJ900	320
2	1	ERJ142	250
3	1	[NULL]	570
4	2	767-301ER	130
5	2	A321	505
6	2	[NULL]	635
7	4	767-301ER	780
8	4	[NULL]	780
9	5	767-301ER	430
10	5	ERJ142	240
11	5	[NULL]	670
12	7	767-301ER	430
13	7	[NULL]	430
14	8	A321	465
15	8	[NULL]	465
16	9	767-301ER	380
17	9	CRJ900	390
18	9	[NULL]	770
19	10	A321	135
20	10	[NULL]	135
21	11	767-301ER	930
22	11	ERJ142	295
23	11	[NULL]	1,225
24	13	A321	395
25	13	[NULL]	395
26	14	767-301ER	170

14. **Create a view for Business class customers**

```
SELECT c.customer_id, c.first_name, c.last_name, t.brand
FROM customer c
JOIN ticket_details t ON c.customer_id = t.customer_id
WHERE t.class_id = 'Business';
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.056s
Start time	Tue Jun 09 06:53:06 SAST 2026
Finish time	Tue Jun 09 06:53:06 SAST 2026
Query	CREATE VIEW business_class_view AS SELECT c.customer_id, c.first_name, c.last_name, t.brand FROM customer c JOIN ticket_details t ON c.customer_id = t.customer_id WHERE t.class_id = 'Business'

15. **Create a stored procedure for passengers in a route range**

```
DELIMITER //
CREATE PROCEDURE GetPassengersByRoute(IN start_route INT, IN end_route INT)
BEGIN
    DECLARE table_count INT;
    SELECT COUNT(*) INTO table_count
    FROM information_schema.tables
    WHERE table_schema = DATABASE() AND table_name = 'passengers_on_flights';

    IF table_count = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Table passengers_on_flights does not exist';
    ELSE
        SELECT * FROM passengers_on_flights
        WHERE route_id BETWEEN start_route AND end_route;
    END IF;
END //
DELIMITER ;
```

Statistics 1

Name	Value
Updated Rows	-1
Execute time	0s
Start time	Tue Jun 09 06:54:01 SAST 2026
Finish time	Tue Jun 09 06:54:01 SAST 2026
Query	DELIMITER ;

16. **Create a stored procedure for long-distance routes (>2000 miles)**

```
DELIMITER //
CREATE PROCEDURE GetLongDistanceRoutes()
BEGIN
    SELECT * FROM routes WHERE Distance_miles > 2000;
END //
DELIMITER ;
```

Statistics 1

Name	Value
Updated Rows	-1
Execute time	0s
Start time	Tue Jun 09 06:54:42 SAST 2026
Finish time	Tue Jun 09 06:54:42 SAST 2026
Query	DELIMITER ;

17. **Create a stored procedure to group flight distances (SDT, IDT, LDT)**

```
DELIMITER //
CREATE PROCEDURE GroupFlightDistances()
BEGIN
    SELECT Flight_num, Distance_miles,
           CASE
               WHEN Distance_miles BETWEEN 0 AND 2000 THEN 'SDT'
               WHEN Distance_miles > 2000 AND Distance_miles <= 6500 THEN 'IDT'
               WHEN Distance_miles > 6500 THEN 'LDT'
           END AS Distance_Category
    FROM routes;
END //
DELIMITER ;
```

Statistics 1

Name	Value
Updated Rows	-1
Execute time	0s
Start time	Tue Jun 09 06:55:41 SAST 2026
Finish time	Tue Jun 09 06:55:41 SAST 2026
Query	DELIMITER ;

18. **Create a function and procedure for complimentary services**

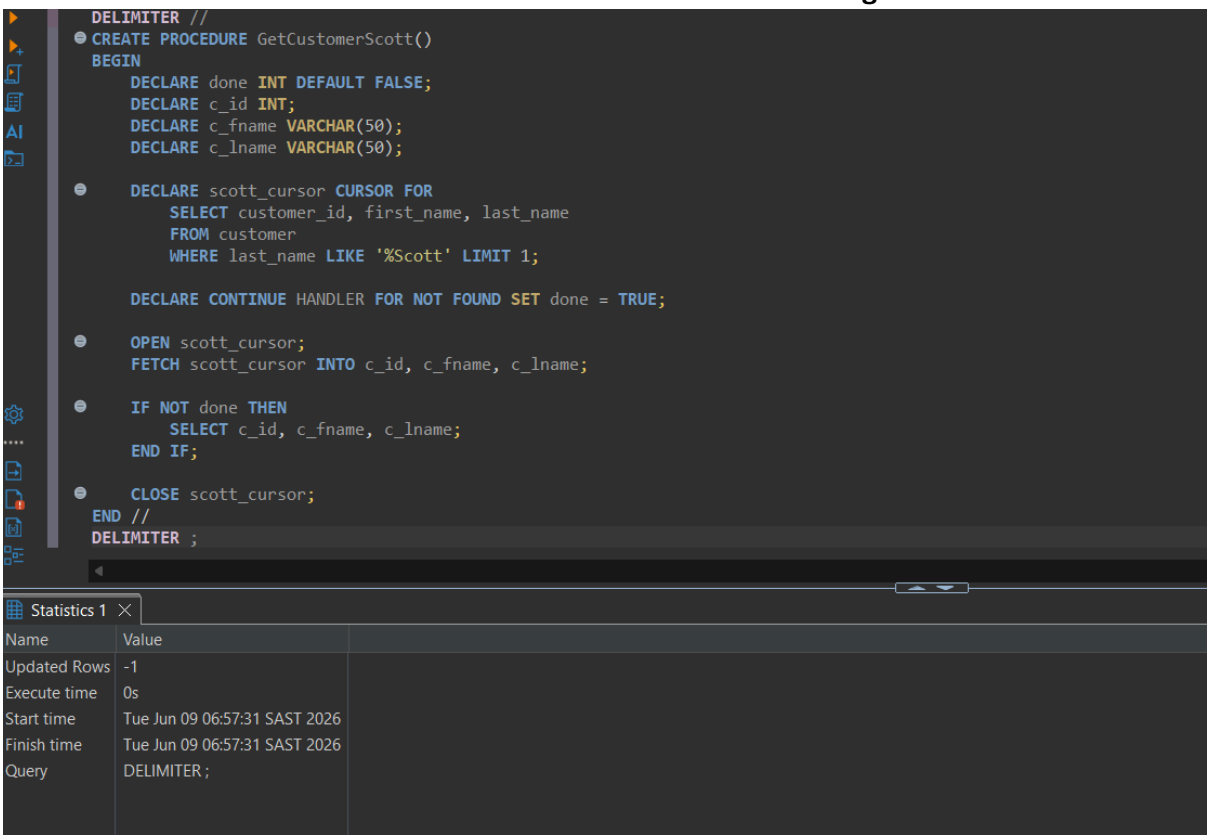
```
DELIMITER //
CREATE FUNCTION CheckComplimentary(class_id VARCHAR(50))
RETURNS VARCHAR(3) DETERMINISTIC
BEGIN
    DECLARE is_complimentary VARCHAR(3);
    IF class_id IN ('Business', 'Economy Plus') THEN
        SET is_complimentary = 'Yes';
    ELSE
        SET is_complimentary = 'No';
    END IF;
    RETURN is_complimentary;
END //

CREATE PROCEDURE GetComplimentaryDetails()
BEGIN
    SELECT p_date, customer_id, class_id,
           CheckComplimentary(class_id) AS complimentary_services
    FROM ticket_details;
END //
DELIMITER ;
```

Statistics 1

Name	Value
Updated Rows	-1
Execute time	0s
Start time	Tue Jun 09 06:56:32 SAST 2026
Finish time	Tue Jun 09 06:56:32 SAST 2026
Query	DELIMITER ;

19. **Extract the first customer whose last name ends with 'Scott' using a Cursor**



```
DELIMITER //
CREATE PROCEDURE GetCustomerScott()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE c_id INT;
    DECLARE c_fname VARCHAR(50);
    DECLARE c_lname VARCHAR(50);

    DECLARE scott_cursor CURSOR FOR
        SELECT customer_id, first_name, last_name
        FROM customer
        WHERE last_name LIKE '%Scott' LIMIT 1;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN scott_cursor;
    FETCH scott_cursor INTO c_id, c_fname, c_lname;

    IF NOT done THEN
        SELECT c_id, c_fname, c_lname;
    END IF;

    CLOSE scott_cursor;
END //
DELIMITER ;
```

Statistics 1

Name	Value
Updated Rows	-1
Execute time	0s
Start time	Tue Jun 09 06:57:31 SAST 2026
Finish time	Tue Jun 09 06:57:31 SAST 2026
Query	DELIMITER ;

Conclusion

This project successfully designed and implemented a robust relational database solution for Air Cargo. By utilizing advanced SQL querying techniques—including window functions, stored procedures, and complex joins—we effectively extracted key insights regarding customer demographics, route traffic, and revenue generation. The establishment of a clean Entity-Relationship model and automated views ensures that Air Cargo can continuously monitor business performance. Ultimately, these data-driven deliverables provide the operational clarity required to optimize fleet deployment, tailor premium customer offerings, and solidify Air Cargo as a customer-centric aviation provider.